
★ 1. Introduction

Definition of an AI, Major Areas of Artificial Intelligence, AI Techniques, History, AI problems, Production Systems, Problem characteristics, Intelligent Agents, Agent Architecture, AI Application (E-Commerce, & Medicine), AI Representation, Properties of internal representation, Future scope of AI, Issues in the design of search algorithms, Introduction to AI Problems and Applications, Defining Problems as a State Space Search, Problem Characteristics, Production Systems

2. Search Techniques:

Generate-And-Test, Hill Climbing, Best-First Search, Problem Reduction, Constraint Satisfaction, Means-Ends Analysis. Heuristic search, mean and end analysis, Constraint Satisfaction, A* and AO* Algorithm, Knowledge Representation: Basic concepts, Knowledge representation Paradigms, Propositional Logic, Inference Rules in Propositional Logic, Knowledge representation using Predicate logic, Predicate Calculus, Predicate and arguments, ISA hierarchy, Frame notation, Resolution, Natural Deduction.

3. Knowledge Representation:

Knowledge Representation – Representation and Mappings, Different Approaches, Issues in knowledge representation. Predicate Logic - Representation Simple Facts in Logic, Representing Instance and Isa Relationships, Computable Functions and Predicates, Resolution. Propositional Logic: Representation, Inference, Reasoning Patterns, Resolution, First-order Logic: Representation, Inference, Reasoning Patterns, Resolution.

4. Uncertainty:

Non-Monotonic Reasoning, Logics for Non-Monotonic Reasoning, Forward rules, and Backward rules, Justification based Truth Maintenance Systems, Semantic Nets Statistical Reasoning, Probability and Bayes' theorem, Bayesian Network, Markov Networks, Hidden Markov Model, Basis of Utility Theory, Utility Functions.

5. Fuzzy Sets and Fuzzy Logic:

Fuzzy Set Operations, Membership Functions, Fuzzy Logic, Hedges, Fuzzy Proposition and Inference Rules, Fuzzy Systems.

6. Natural Language Processing:

Introduction, Syntactic Processing, Semantic Analysis, Semantic Analysis, Discourse and Pragmatic Processing, Spell Checking.

7. Neural Networks and Expert systems:

Introduction to neural networks and perception-qualitative Analysis, Neural net architecture and applications, Utilization and functionality, the architecture of the expert system, knowledge representation, two case studies on expert systems.

Unit 1 : Introduction

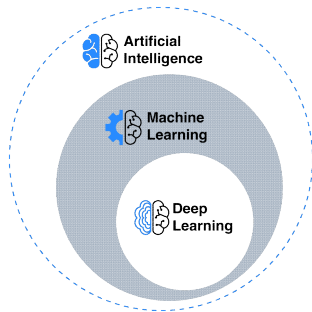
06 July 2026 18:12



What is Artificial Intelligence ?

AI is a branch of Computer science that deals with the design and development of Intelligent machines that can perform task requiring human Intelligence. These tasks include :

- Learning : learn from the data and improve their performance over time.
- Reasoning : AI can analyze information and make logical decisions
- Problem-solving : find solutions for simple and complex problems
- Decision making : select best possible actions based on the available information
- Understanding :
- Recognizing patterns
- Natural language



Artificial Intelligence Techniques:

AI techniques are method and approaches used to develop intelligent systems that can solve problems, learn from experience, reason and make decisions like human

Search Techniques:

- Search techniques are used to find best solutions from the many possible solutions by exploring different states of problem
- AI techniques avoid checking every possible solution. They use heuristic to search only the most promising solutions.
- This saves both time and memory

Reasoning

- is a process of drawing a conclusion from available knowledge and facts to solve a problem.
- They compare different alternatives before selecting the best one. This improves decision quality.

Knowledge representation:

- It is a process of storing facts, relationships and rules in a form that a computer understand and use it when reasoning

Handles incomplete or uncertain information

- AI can make decisions even when all information is not available. Is estimate the most likely solutions using available knowledge.
- This is useful in real-world situations ex: weather forecasting based on complete weather data



Major Areas of Artificial Intelligence

The major areas of AI is :

- **Machine Learning**
ML is a branch of AI that enable Computers to learn from data without being explicitly programmed. It improves performance by analyzing past experience and patterns. ML algorithms are trained by large datasets
Supervised learning : involves learning from labeled data , where each data point have corresponding label
Unsupervised learning: deals with unlabeled data. Where algorithm identify the patterns and relationships in data on its own
Reinforcement learning: it involves an agent interacting with environment and learn from trial and error method. The agent receives reward for desired actions and penalties for undesired actions
Ex: Spam Mail Detection , Recommendation System (Netflix , amazon)
- **Natural Language Process(NLP)**
NLP enables computers to understand , interpret and generate human language in both text and speech form
Software could now response creatively, AI agents with NLP can summarize meetings, draft emails and translate conversions in real-time
NLP combines computational linguistics , predictive Artificial intelligence and deep learning models to process human language.
Ex: Chatbot , Language translation, voice assistance, sentiment analysis
- **Computer vision**
it enables machines to capture , process and understand images and videos, similar to human vision
It uses deep learning and neural networks to recognize objects , people , patterns
Computer vision can run on cloud , on-premises and on edge devices. This versatility drive efficiency and innovation
Ex: Face recognition , medical image analysis, self-driving car , object detection
- **Robotics**
it combines AI with mechanical engineering to develop intelligent robots that can perform tasks automatically or with minimal human instruction
Ex: Industrial Automation, medical surgery robots Wherehouse robots



Artificial Intelligence Problems

- AI problem is a problem that requires machines to perform tasks that are normally needed human intelligence such as reasoning, decision making, learning , understanding patterns.
- AI problems are difficult because it involves large amount of knowledge , uncertainty and many possible solutions

Problem of knowledge representation and complexity

- AI system need to store a huge amount of knowledge about real world
- The representation should be represented in form that computer can understand and use. Poor knowledge representation leads to incorrect decisions.
- Vast amount of data and large relationships with variable and a high degree of uncertainty . This complexity makes It challenging to find the optimal solutions.

Problem of search

- Many AI problems have a very large number of possible solutions. AI must search efficiently to find the best solution
- Searching every possibility is often impossible because it takes to much time. AI uses search algorithms and heuristics to reduce the search space.

Uncertainty and ambiguity

- Real world data is often messy and incomplete . Sensors reading can be noisy and information might have multiple interpretations.

Problem of reasoning

- AI must think logically before making decisions.
- It should draw correct conclusions from the available information. Often information is incomplete or uncertain , making reasoning difficult.

Problem of Natural language Understanding (NLP)

- Humans communicate using natural language like English, Hindi, Telugu. AI finds it difficult to understand grammar, meaning and context.
- The same word may have different meanings in different situations. AI must understand , process and generate human

it combines AI with mechanical engineering to develop intelligent robots that can perform tasks automatically or with minimal human instruction

Ex: Industrial Automation, medical surgery robots Warehouse robots

- **Expert systems**

Expert systems are AI programs that imitate decision-making ability of human experts by using a knowledge base inference rules

Ex: Medical diagnosis , Financial advisory systems, Troubleshooting system



Production System

A production system is a rule based of computation used in Artificial Intelligence (RULE BASED MODEL: system where knowledge and decision- making are encoded explicitly , a collection of rules : IF fever AND cough THEN flu) To solve problems by applying a sequence of production rules.

Components of Production Systems:

- **Production rules**
Production rules are IF-THEN(conditional statements) the condition specifies when the rule is applicable
Ex(IF fever AND cough THEN flu)
- **Working memory**
Working memory stores the current state of the problem. It contains facts and information about the current situation.
Whenever a rule is executed the contents of working memory will be updated.
- **Control System**
Decides which rule to apply based on the content facts.
Applies rules iteratively until a solution or goal stat is reached.
- **Conflict resolution**
Sometimes multiple rules satisfies the current condition. conflict resolution determines which rule should be executed.

Application of Production System

- Expert systems.
- Medical diagnosis
- Automated reasoning
- Robotics
- Intelligent tutoring system
- Rule based decision support system



Defining Problems as a State Space Search

In Artificial Intelligence, many problems are solved by representing them as a **State Space Search**. A state space consists of all possible situations (states) that can occur while solving a problem. The AI system starts from an **initial state** and explores different states until it reaches the **goal state** by applying a sequence of valid actions (operators)

State Space Search

is a problem-solving technique in Artificial Intelligence in which a problem is represented as a collection of states connected by operators, and a search algorithm explores these states to find a path from the initial state to the goal state.

Components of State Space Search

A problem must be clearly defined before it can be solved using state space search.

- **1. Initial State**
The initial state represents the starting condition of the problem before any action is performed.
It is the point from which the search process begins.
- **Goal State**
The goal state is the desired final condition that the AI system aims to reach.
The search process terminates once the goal state is found.
- **States**
A state represents the current condition of the problem at any stage during the search.
Each action changes the current state into another state.



AI Representation

AI helps in understanding the meaning and context.

- The same word may have different meanings in different situations. AI must understand , process and generate human language



Problem Characteristics

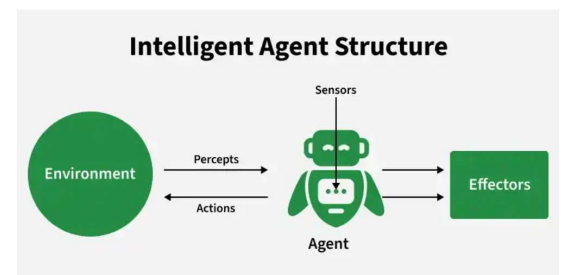
Problem characteristics is a property of the problem that helps in selecting the most suitable Artificial Intelligence technique or search strategy.

- **Uncertainty And Ambiguity**
Real-world information is often incomplete , noisy or uncertain. AI Systems must make decisions even when all information is not available
- **Lack of clear problem Definition**
Sometimes the problem itself might not be well-defined.
- **Complexity**
Many AI problems involve vast amounts of data, many relationships between variables, and a high degree of uncertainty.



Intelligent Agents

- Intelligent agents is an autonomous Software program that observe its environment through sensors, processes the information , makes intelligent decisions and performs actions through physical actions to achieve specific goals.
- Unlike Ordinary programs , intelligent agents can reason , learn, adapt and interact with their environment to improve their performance over time.
- AI systems such as self-driving cars, virtual assistants , robots and recommendations systems are examples of intelligent agents
- **Perception:** intelligent agents can gather information about their environment through sensors. This can include cameras, LIDAR (Light detection and ranging) for robots or microphones for virtual assistance.
- **Learning:** they can improve their performance over time by learning from past experiences or data. This could involve machine learning algorithms algorithms.
- **Adaptability:** intelligent agents cab adjust their behavior based on new information or changing circumstances.



AI Application (E-commerce & medicine)

AI application in E-commerce

- **Personalized product recommendation**
AI analyzes the user browsing history , purchase history and preferences to recommend the products that customers likely to buy.
EX(Amazon and Flipkart recommend products based on recent purchase history)
- **Chat box and virtual assistance**
AI power chat-box provide 24x7 customer support by answering questions , track orders, handling complaints and assisting customers during shopping.
- **Fraud detection**
AI detect suspicious transactions by analyzing customer



AI Representation

- AI representation, also called Knowledge representation (KR) or internal representation, is the method of organizing and storing knowledge inside an Artificial Intelligence system so that it can understand, reason, learn, and solve problems.
- Knowledge representation acts as the bridge between the real world and AI system. It enables AI to represent facts, objects, events, relationships, and rules in a structure form that can be processed by computer. Without proper representation, an AI cannot make intelligent decisions or perform reasoning effectively
- Knowledge Representation is a technique used to store knowledge in a computer so that it can think, reason, and make intelligent decisions.

Techniques of Knowledge Representation

There are several methods used to represent knowledge in AI.

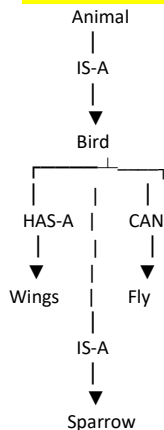
- Logical Representation**

Logical representation uses propositional logic and predicate logic to represent knowledge.

Example:

All humans are mortal.
Socrates is a human.
Therefore,
Socrates is mortal.

- Semantic Network**



- Production Rules**
Production rules in AI are a group of rules that apply to information collected from a global database. Each rule has a precondition and a postcondition that the global database must either fulfill or not, depending on the rule.
- Frames Representation**
A frame-based expert system is an (AI) program that uses a knowledge representation technique to store and reason with facts about the world. It works on the basis of frames, which are structures designed to capture relevant information from a particular domain such as medicine or business.

Applications of Knowledge Representation

- Expert Systems
- Medical Diagnosis
- Robotics
- Natural Language Processing (NLP)
- Question Answering Systems
- Intelligent Tutoring Systems
- Machine Translation
- Planning Systems
- Decision Support Systems



Future scope of Artificial Intelligence

Future Scope of Artificial Intelligence refers to the potential growth, advancements, and applications of AI in different fields that will improve human

searching questions, tasks, and managing complexities and assisting customers during shopping.

- Fraud detection**
AI detect suspicious transactions by analyzing customer behavior and payment patterns
- Inventory management**
AI predicts product demand and manage stock levels and reduce overstocking or shortages
This help business improve warehouse efficiency
- Dynamic pricing**
AI adjust price automatically based on demand, competition, customer, behavior, and market conditions

AI application in Medicine

Artificial intelligence has greatly improved healthcare by supporting doctors in diagnosis, treatments, planning and patient care

- Personalized Medicine**
AI analyze a patient's genetic information, lifestyle, and medical history to recommend treatments that specifically suited to that individual. This increases treatments effectiveness and reduces side effects.
- Early Disease Diagnosis**
Machine Learning algorithms analyze medical images, laboratory reports and genetic data to detect disease such as cancer, diabetes and heart disease at an early stage.
- Drug Discovery**
AI accelerates the process of developing new medicines by predicting how different chemical compounds interact with diseases, reducing both research time and development cost.
- Medical image Analysis**
AI analyzes X-rays, MRI scans and ultrasound images to identify abnormalities with high accuracy, helping radiologists make better decisions
- Patient Monitoring**
AI continuously monitors patients using wearable devices and hospital equipment, Providing early when abnormal conditions are detected.



Internal representation

Internal Representation (also called **Knowledge Representation**) refers to the way knowledge is stored inside an AI system so that it can understand, reason, and solve problems

- Representational Adequacy**
Representational adequacy is the ability of a knowledge representation scheme to represent all the necessary facts, objects, events, relationships, and rules of a particular domain.
An AI system should be capable of representing different types of knowledge such as facts, procedures, relationships, and constraints without losing information.
- Inferential Adequacy**
Inferential adequacy is the ability of the representation to generate new knowledge from the existing knowledge through logical reasoning.
The system should not only store facts but also derive conclusions automatically.
Knowledge stored:
Birds can fly.
Parrot is a bird.
Derived knowledge:
Parrot can fly.

- Inferential Efficiency**
Inferential efficiency refers to how quickly and efficiently the AI system can perform reasoning.
A good representation should organize knowledge in such a way that searching and reasoning require minimum time and computational effort.
Example
Suppose a navigation system stores information about thousands of roads. A well-organized representation helps the system quickly find the shortest path instead of searching every possible route.

- Acquisitional Efficiency**
Acquisitional efficiency is the ability to easily add, modify, or remove knowledge from the knowledge base without affecting the existing information.
As knowledge changes over time, the representation should allow convenient updates.



Future scope of Artificial Intelligence

Future Scope of Artificial Intelligence refers to the potential growth, advancements, and applications of AI in different fields that will improve human life, industries, and scientific research in the future.

Healthcare and Medical Diagnosis

Artificial Intelligence is expected to revolutionize the healthcare industry by providing faster and more accurate medical services. AI systems will assist doctors in diagnosing diseases at an early stage, recommending treatments, and monitoring patients continuously.

Future AI applications in healthcare include:

- Early disease detection
- Medical image analysis
- Robot-assisted surgeries
- Personalized treatment plans
- Drug discovery and development
- Virtual health assistants

Education

AI will make education more personalized and interactive. Intelligent tutoring systems will adapt lessons according to each student's learning speed and abilities.

Future applications include:

- Smart classrooms
- Personalized learning
- AI tutors
- Automatic grading
- Virtual laboratories
- Language translation for education

Natural Language Processing (NLP)

Future AI systems will communicate with humans more naturally using spoken and written language.

Applications include:

- AI chatbots
- Language translation
- Voice assistants
- Speech recognition
- Text summarization
- Sentiment analysis

Robotics and Automation

Future robots will become more intelligent, autonomous, and capable of performing complex tasks without human intervention.

Applications include:

- Industrial robots
- Household robots
- Service robots
- Disaster rescue robots
- Agricultural robots
- Space exploration robots

Acquisitional efficiency is the ability to easily add, modify, or remove knowledge from the knowledge base without affecting the existing information.

As knowledge changes over time, the representation should allow convenient updates.

Example

Suppose a university introduces a new course.

The AI academic advisor should easily update its knowledge base by adding the new course without changing all existing student records.

Flexibility

Flexibility means that the representation should support different kinds of knowledge and allow expansion when new information becomes available.

It should adapt to changing environments without requiring a complete redesign.



Issues in the Design of Search Algorithms

Search algorithms play an important role in **Artificial Intelligence (AI)**. They are used to solve problems by exploring different possible states until a goal state is reached. Search algorithms are widely applied in **planning, game playing, robotics, expert systems, machine learning, and route finding**.

→ **Search algorithm design** is the process of developing an efficient method that explores a problem space to find the best or most suitable solution while minimizing time, memory usage, and computational cost.

Major Issues in the Design of Search Algorithms

1. Problem Representation

The first issue in designing a search algorithm is how the problem is represented.

The problem should be represented in terms of:

- Initial state
- Goal state
- Set of possible actions (operators)
- State transitions
- Cost of actions

A good representation reduces the complexity of searching and helps the algorithm find solutions efficiently.

Size of the Search Space

The search space consists of all possible states that can be generated from the initial state.

For many AI problems, the number of possible states grows exponentially, making exhaustive search impractical.

A search algorithm should minimize unnecessary exploration of the search space.

Example : In chess, there are millions of possible board positions. Examining every possible move is computationally impossible, so intelligent search strategies are required.

Time Complexity

Time complexity refers to the amount of time required by the search algorithm to find a solution.

A poorly designed search algorithm may examine thousands or millions of unnecessary states before reaching the goal.

The algorithm should minimize the number of explored nodes while still finding a correct solution.

Space Complexity

Space complexity refers to the amount of memory required to store generated nodes during the search process.

Some algorithms require large amounts of memory because they store all previously generated states.

An efficient search algorithm should use memory carefully without affecting performance.

Loop Detection and Repeated States

Many search problems contain cycles, where the algorithm repeatedly visits the same state.

Without proper loop detection, the algorithm wastes time and memory and may never terminate.

Techniques such as maintaining a **visited list** or **closed list** help avoid revisiting previously explored states.

Unit 2 : Search Techniques

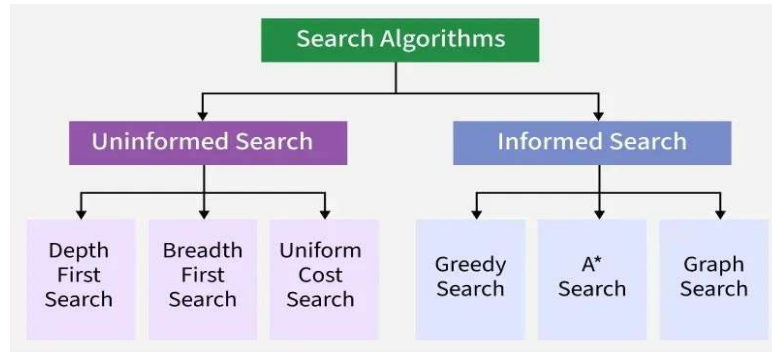
31 July 2026 15:21



Search Algorithms in Artificial Intelligence

A **search algorithm** is a systematic procedure used by an AI system to explore all possible states of a problem in order to find a sequence of actions that leads from the **initial state** to the **goal state**.

Whenever an AI system does not know the direct solution to a problem, it searches through different possible alternatives until it discovers the correct or best solution.



Uninformed Search

Uninformed Search algorithms search for the solution **without any additional knowledge about the location of the goal**.

The algorithm only knows:

- Initial state
- Goal test
- Available operators

It does not know whether a particular state is closer to the goal.

Uninformed search is also called Blind search.



Informed Search

the algorithms have information on the goal state, which helps in more efficient searching. This information is obtained by something called a heuristic. In an informed search, a heuristic is a function that estimates how close a state is to the goal state.

Instead of searching blindly, it intelligently chooses nodes that appear closer to the goal.

What is a Heuristic?

A **heuristic** is a rule, estimate, or intelligent guess that predicts how close a state is to the goal.

It does not guarantee perfection but usually finds solutions much faster.



Generate-And-Test

Generate-and-Test is one of the simplest and oldest problem-solving techniques used in Artificial Intelligence. It works by **generating possible solutions one by one and testing each solution to determine whether it satisfies the goal condition**. If the generated solution is correct, the search stops; otherwise, the algorithm continues generating and testing new solutions until a valid solution is found or all possibilities have been exhausted.

Generate possible solutions → Test each solution → Accept if correct → Otherwise generate another solution.

It is called a **heuristic search method** because the generation process can be guided using heuristic knowledge to reduce the number of unnecessary solutions that need to be tested. Without heuristics, it behaves like brute-force search.

Working principle

The generate and test algorithms works in the following steps

Step 1: Generate a Candidate Solution

The algorithm first creates a possible solution.

This solution may be:

- Randomly generated
- Systematically generated
- Generated using heuristic knowledge

Step 2: Test the Solution

The generated solution is checked against the goal condition.

Questions asked include:

- Does it satisfy all constraints?
- Is it the desired goal?
- Is it a valid solution?

Step 3: Accept or Reject

- If the solution satisfies the goal, the algorithm terminates.
- Otherwise, it rejects the solution.

Step 4: Generate Another Solution

The algorithm again generates another candidate and repeats the testing process.

Step 5: Continue Until Completion

The process continues until:

- A solution is found, or
- All possible solutions have been exhausted.



Heuristic search

Heuristic Search is a search strategy that uses **heuristic information** to decide which state should be explored next. Instead of examining every possible path like blind search, it estimates which path is more likely to reach the goal quickly.

A heuristic search algorithm tries to make **intelligent decisions** by selecting the most promising state at every step

A **heuristic** is a rule, estimate, or educated guess that predicts **how close a current state is to the goal state**.

A heuristic does not always guarantee the best solution, but it usually finds a good solution much faster than blind search.

Working Principle of Heuristic Search

The heuristic search process consists of the following steps:

Step 1: Start from the Initial State

The algorithm begins with the given starting state.

Step 2: Evaluate Neighbouring States

All possible successor states are generated.

Each state is assigned a **heuristic value $h(n)$** that estimates how close it is to the goal.

Step 3: Choose the Best State

The state with the most promising heuristic value is selected for further exploration.

Step 4: Repeat the Process

Again, new neighbouring states are generated and evaluated.

The search continues until the goal state is reached.

Step 5: Stop

When the goal state is found, the algorithm terminates.

Real-Life Example: Google Maps

A practical example of heuristic search is **Google Maps**.

Suppose you want to travel from your home to your college.

Google Maps does not check every possible road in the city.

Instead, it estimates which roads are likely to lead you to your destination by considering:

- Straight-line distance
- Current traffic
- Road conditions
- Speed limits
- Estimated travel time

It then recommends the route that appears to reach the destination most quickly.

Step 5: Continue Until Completion

The process continues until:

- A solution is found, or
- All possible solutions have been tested.



Hill climbing

Hill Climbing is a **heuristic search algorithm** that starts from an initial state and repeatedly moves to the neighbouring state with the **best heuristic value**. The objective is to improve the current solution step by step until the goal is reached or no further improvement is possible.

It is called **Hill Climbing** because it is similar to a person climbing a hill. At every step, the climber always moves upward toward a higher point. When there is no higher point available, the climber stops.

Unlike blind search, Hill Climbing does not explore all possible paths. It only focuses on the **most promising neighbouring state**, making the search much faster.

Working Principle

The Hill Climbing algorithm works as follows:

Step 1: Start with the Initial State

The algorithm begins from the given starting state.

Step 2: Generate Neighbouring States

All possible neighbouring states of the current state are generated.

Step 3: Evaluate Each Neighbour

A **heuristic function** is used to measure the quality of each neighbouring state.

The heuristic tells how good or close each state is to the goal.

Step 4: Select the Best Neighbour

The algorithm compares all neighbouring states and chooses the one with the **highest evaluation (or lowest cost, depending on the problem)**.

If the selected neighbour is better than the current state, move to it.

Step 5: Repeat

Again generate neighbours of the new state and repeat the evaluation process.

Step 6: Stop

The algorithm terminates when:

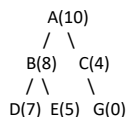
- The goal state is reached, or
- No neighbouring state is better than the current state.

→ Example

Route Finding

Suppose we want to travel from **A** to **G**.

Each city has a heuristic value showing its estimated distance to the goal.



Smaller heuristic value means closer to the goal.

Step 1

Current state

A (10)

Neighbours

B (8)

C (4)

Choose

C

because it is closer to the goal.

Step 2

Current state

C (4)

Neighbour

G (0)

Move to G.

Goal reached.

Problem Reduction

Problem Reduction is a heuristic search technique in which a complex problem is divided into several smaller and simpler subproblems. Instead of solving the entire problem at once, the algorithm solves each subproblem individually and combines their solutions to obtain the final solution.

Each subproblem is solved separately, and their solutions are combined to solve the original problem.

The main objective of Problem Reduction is to reduce the complexity of the search process.

- Road conditions
- Speed limits
- Estimated travel time

It then recommends the route that appears to reach the destination most quickly. Thus, Google Maps uses heuristic information to guide the search efficiently.



Best-First Search

This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search. BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level. The breadth-first search algorithm is an example of a general-graph search algorithm. Breadth-first search implemented using FIFO queue data structure.

Working Principle of BFS

1. Initialization

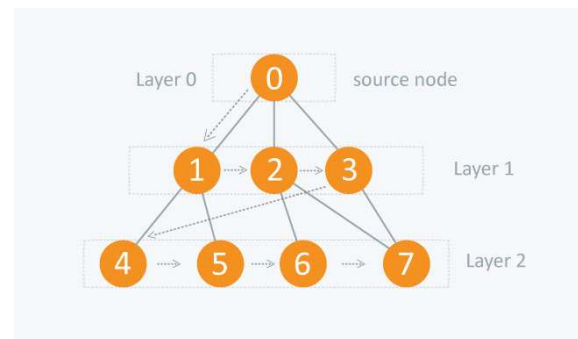
- Choose a starting node (called the **source**).
- Mark it as **visited** to avoid revisiting.
- Place it into a **queue** (FIFO structure).

2. Queue Processing

- While the queue is not empty:
 - Remove (dequeue) the front node.
 - Visit all its **adjacent unvisited neighbours**.
 - Mark each neighbour as visited.
 - Insert (enqueue) each neighbour into the queue.

3. Level-by-Level Expansion

- BFS explores nodes in **layers (levels)**:
 - First, all nodes at distance 1 from the source.
 - Then, all nodes at distance 2, and so on.
 - This ensures that the traversal order corresponds to the shortest distance from the source.
- ##### 4. Termination
- The algorithm stops when the queue becomes empty, meaning all reachable nodes have been visited.



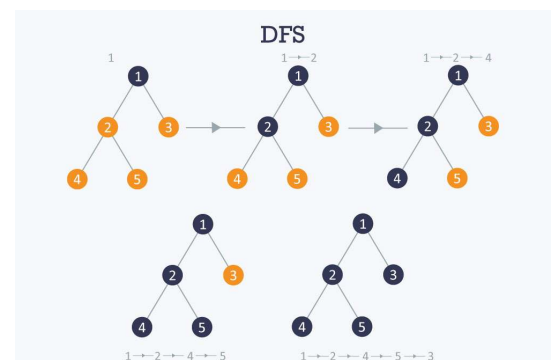
0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7

Depth First Search

The DFS algorithm is a recursive algorithm that uses the idea of backtracking. It involves exhaustive searches of all the nodes by going ahead, if possible, else by backtracking.

Here, the word backtrack means that when you are moving forward and there are no more nodes along the current path, you move backwards on the same path to find nodes to traverse.

It uses a **stack** (explicit or via recursion) to remember the path, making it suitable for exploring deep structures like trees and graphs.



Can the student ride the bike? The algorithm checks.
It finds: Bike has no fuel.
The operator cannot be applied.
Step 4: Create Subgoal
New Subgoal: Fill fuel in the bike.
Step 5: Solve the Subgoal
The student goes to the petrol station.
Fuel is filled. Subgoal completed.
Step 6: Apply Original Operator
Now, Ride Bike becomes possible.
Step 7: Reach Goal
The student rides the bike and reaches college.

selected path.
A* is mainly used for **path-finding and shortest-route problems**.

The implementation of A* is comparatively **simple** because it deals only with ordinary graph structures.

Examples of applications include **Google Maps, GPS navigation, robot navigation, and game path finding**.

the current best solution graph.
AO* is mainly used for **problem reduction, planning, theorem proving, and expert systems**.

AO* is **more complex** because it must manage AND-OR relationships and repeatedly update heuristic costs.

Examples of applications include **robot planning, automated planning, medical diagnosis, expert systems, and theorem proving**.



AO* Algorithm

AO* (AO-Star) is a **heuristic search algorithm** used for solving problems represented as **AND-OR graphs**. It searches for the **minimum-cost solution graph** by selecting the most promising nodes based on heuristic estimates.
Unlike A*, which finds the shortest path from the start node to the goal node, AO* finds the **best combination of subproblems** that leads to the solution.

What is an AND-OR Graph?

An **AND-OR graph** is a graphical representation used to solve complex problems. It contains two types of nodes.

1. OR Node

An OR node represents **alternative choices**.
Only **one** child node needs to be selected.

2. AND Node

An AND node represents **dependent subproblems**.
All child nodes must be solved.

Working Principle of AO*

The AO* algorithm works through the following steps.

Step 1: Start with the Initial Node

The algorithm begins from the root node.
Initially, heuristic values are assigned to every node.

Step 2: Select the Best Partial Solution

Using heuristic values, AO* selects the most promising branch.
This branch is called the **current solution graph**.

Step 3: Expand the Selected Node

The selected node is expanded.
Its successor nodes are generated.

Step 4: Calculate Cost

The cost of newly generated nodes is calculated.
For:

- **OR node** → Select the child having the minimum cost.
- **AND node** → Add the costs of all required child nodes.

Step 5: Update Parent Nodes

After computing the costs, the algorithm updates the cost values of all parent nodes.
The best solution graph is modified accordingly.

Step 6: Repeat the Process

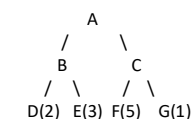
The algorithm again selects the most promising incomplete node and expands it.
This process continues until every node in the solution graph becomes a goal node.

Step 7: Obtain Final Solution

When every node in the selected solution graph is solved, the algorithm terminates.
The final solution graph is returned.

Example

Suppose we have the following AND-OR graph.



Assume:

- A is an **OR node**.
- B is an **AND node**.
- C is an **OR node**.

Step 1

Start from node A.
There are two choices:
Go to B / Go to C



A* algorithm

A* (A-Star) Search is a heuristic search algorithm that finds the shortest path from the initial state to the goal state by evaluating each node using both the **actual path cost** and the **estimated remaining cost**.

The A* Search Algorithm is an optimal and efficient method for solving pathfinding problems. When seeking the **shortest path** between a starting point and a goal, A* is often the best choice due to its versatility and robustness.

Formula: $f(n) = g(n) + h(n)$

Working Principle

The A* Search algorithm works through the following steps.

Step 1: Start with the Initial Node

The algorithm begins from the given start node.
The start node is inserted into the **OPEN list**.

Step 2: Compute Evaluation Function

For every generated node,
Calculate: $f(n) = g(n) + h(n)$

Step 3: Select the Best Node

From the OPEN list, select the node having the **smallest f(n)**.
This node is considered the most promising.

Step 4: Check Goal

If the selected node is the goal, the algorithm terminates.
Otherwise, continue searching.

Step 5: Expand the Node

Generate all neighbouring nodes.
Calculate : $g(n)$, $h(n)$, $f(n)$
for every neighbour.

Step 6: Update OPEN and CLOSED Lists

The expanded node is moved to the CLOSED list.
New nodes are inserted into the OPEN list.

Step 7: Repeat

Again select the node having the lowest $f(n)$
Continue until the goal is reached.

Advantages of A*

- **Optimality:** A* guarantees finding the shortest path between a given start and goal node, provided that certain conditions are met
- **Efficiency:** A* is typically more efficient than brute-force search algorithms like breadth-first search or depth-first search, especially in large graphs.
- **Flexibility:** A* can be applied to a wide range of problems, including pathfinding in grids, maps, or networks
- **Speed:** A* is often faster than other uninformed search algorithms because it intelligently prioritizes the exploration of nodes.

Disadvantages of A*

- **Heuristic Accuracy:** The performance of A* heavily relies on the accuracy of the heuristic function used
- **Memory Requirements:** A* needs to keep track of the nodes in the open and closed sets during the search process.
- **Computational Complexity:** Although A* is generally efficient, its worst case time complexity is exponential. In graphs with many possible paths or when using a poor heuristic, A* can degrade in performance. In such cases, alternative algorithms like Dijkstra's algorithm (which guarantees optimality but lacks the heuristic-guided search) might be more suitable.
- **Path Preoptimization:** A* assumes that the environment and costs remain static throughout the search process

Start from node A.
There are two choices:
Go to B / Go to C

Step 2

Evaluate node B.
Since B is an AND node,
both children must be solved.
Cost: $B = D + E$
 $= 2 + 3 = 5$

Step 3

Evaluate node C.
Since C is an OR node,
only one child is required.
Choose the smaller cost.
 $F = 5$
 $G = 1$
Therefore,
 $C = 1$

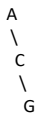
Step 4

Compare the two alternatives.
Path through B = 5
Path through C = 1
Since

$1 < 5$
AO* selects

$A \rightarrow C \rightarrow G$

Final Solution Graph



Production Rules

Production rules are like "if-then" instructions.

- The **condition part** says *when* the rule should apply (for example: *if the light is red*).
- The **action part** says *what to do* when the condition is true (for example: *then stop the car*). They are useful for representing **step-by-step knowledge** about how to solve problems or perform tasks.

Knowledge Base

A knowledge base is basically a **storage place for knowledge**. It can be:

- A database,
- A collection of rules,
- An ontology (structured concepts and relationships),
- Or a mix of these.

Think of it as the **brain of an intelligent system**, where all the facts and rules are kept.



Propositional Logic

Propositional Logic (also called **Statement Logic** or **Sentential Logic**) is the simplest form of formal logic used in Artificial Intelligence. It represents knowledge using **propositions (statements)** that are either **True (T)** or **False (F)**, but never both at the same time.

→ Logical Connectives

Logical connectives combine simple propositions into complex propositions.

1. Negation (NOT)

Symbol: $\neg P$

Meaning: Negation reverses the truth value.

"If P is true, then NOT P is false."

Example

P: It is raining.

NOT P: It is not raining.

2. Conjunction (AND)

Symbol: $\wedge$

Meaning: Both statements must be true.

Example

P: I study AI.

Q: I study Python.

Compound statement: I study AI AND Python. ($P \wedge Q$)

3. Disjunction (OR)

Symbol: $\vee$

$P \vee Q$

Meaning: At least one statement must be true.

Compound statement: I will study AI OR Python.

Types of Knowledge Representation Paradigms

1. Simple Relational Knowledge

- It is the simplest method of knowledge representation.
- Knowledge is represented in the form of **relations or tables**, similar to database systems.
- Information is stored as rows and columns, making it easy to organize and retrieve facts.
- It is suitable for representing simple factual information but cannot efficiently represent complex relationships or reasoning.
- This paradigm is widely used in database management systems and information storage applications.

Example:

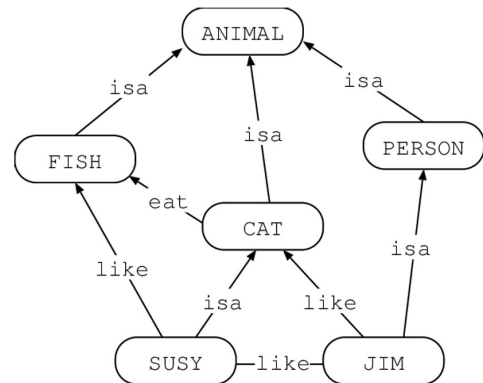
Student Course Grade

Rahul AI A

Priya DBMS B+

2. Inheritable Knowledge

- In this paradigm, knowledge is organized into **hierarchical structures** using **IS-A** and **Instance-Of** relationships.
- Child objects automatically inherit the properties of their parent objects.
- It avoids duplication of knowledge because common properties are stored only once at a higher level.
- Semantic Networks and Frames commonly use inheritance mechanisms.
- This paradigm is useful for representing classification and taxonomic knowledge.



3. Inferential Knowledge

- Inferential knowledge represents facts together with logical rules that allow new knowledge to be derived from existing facts.
- It uses logical inference techniques such as **predicate logic** and **production rules**.
- Instead of storing every fact explicitly, the system can infer additional information automatically.
- This paradigm improves reasoning capability and supports intelligent decision-making.
- It is widely used in expert systems and automated reasoning.

Example:

Facts:

All humans are mortal.

Socrates is a human.

Inference:

Therefore,

Socrates is mortal.

The final statement is not stored directly but is inferred using logical reasoning.

4. Procedural Knowledge

- Procedural knowledge represents **how to perform a task** rather than simply describing facts.
- Knowledge is stored in the form of procedures, algorithms, rules, or programs.
- It specifies the sequence of operations required to solve a problem.
- This paradigm is useful when the order of execution is important.
- It is commonly used in expert systems, robotics, and planning systems.

Example:

Making Tea:

Boil water

↓

Add tea powder

↓

Add milk

↓

Add sugar

↓

Serve tea

The emphasis is on the sequence of actions rather than factual information.

Example: I will study AI OR Machine Learning.

4. Implication (IF...THEN)

Symbol: $\rightarrow$

Meaning: "If P is true, then Q must also be true."

Example: If it rains, then roads become wet.

$P = \text{It rains}; Q = \text{Roads become wet} \quad (P \rightarrow Q)$

5. Biconditional (IF AND ONLY IF)

Symbol $\leftrightarrow$; $P \leftrightarrow Q$

Meaning: Both statements should have the same truth value.

Example : You pass the exam if and only if you score above 40 marks.



Predicate Calculus, Predicate and arguments

Predicates: Predicates represent properties or relationships between objects.

• **Statements:** 1. Animal(Dog): This statement asserts that Dog is an animal.

• **Loves(John, Mary):** This statement asserts that John loves Mary

• **Universal Quantifier:**

• $\forall x \text{ Animal}(x)$: This statement asserts that "For all x, x is an animal." It implies that every entity in the domain is an animal.

• **Existential Quantifier:**

• $\exists x \text{ Loves}(x, \text{Dog})$: This statement asserts that "There exists an x such that x loves Dog." It implies that there is at least one entity that loves the Dog.



Knowledge Representation using Predicate Logic

Problem Statement

Consider the following information:

1. All students study at a university.
2. Every student has a valid ID card.
3. Nikhil is a student.
4. Venkat is a student.
5. Every student who studies Artificial Intelligence is an AI student.
6. Nikhil studies Artificial Intelligence.

Represent this knowledge using **Predicate Logic** and infer new facts.

Step 1: Identify Objects (Constants)

Objects represent specific entities.

Nikhil, Venkat, University

Artificial_Intelligence

Step 2: Identify Predicates

Predicates describe properties or relationships.

Student(x) , Studies(x, y) , HasID(x) , AIStudent(x)

UniversityStudent(x)

Meaning:

- Student(x) $\rightarrow$ x is a student.
- Studies(x,y) $\rightarrow$ x studies subject y.
- HasID(x) $\rightarrow$ x has a valid ID card.
- AIStudent(x) $\rightarrow$ x is an AI student.
- UniversityStudent(x) $\rightarrow$ x studies at a university.

Step 3: Represent Facts

→ **Fact 1**

"Nikhil is a student."

Student(Nikhil)

→ **Fact 2**

"Venkat is a student."

Student(Rahul)

→ **Fact 3**

"Nikhil studies Artificial Intelligence."

Studies(Nikhil, Artificial_Intelligence)

Step 4: Represent Rules

→ **Rule 1**

"Every student studies at a university."

$\forall x \text{ Student}(x) \rightarrow \text{UniversityStudent}(x)$

Meaning:

For every person x,

If x is a student,

Then x is a university student.

→ **Rule 2**

"Every student has a valid ID."

$\forall x \text{ Student}(x) \rightarrow \text{HasID}(x)$

Meaning:

Every student possesses a valid identity card.

→ **Rule 3**

"Every student studying Artificial Intelligence is an AI student."

$\forall x \text{ Studies}(x, \text{Artificial_Intelligence})$

$\text{AIStudent}(x)$

Meaning:

If a person studies Artificial Intelligence,
then that person becomes an AI student.

Step 5: Knowledge Base

The complete knowledge base becomes

Student(Nikhil)

Student(Venkat)

Studies(Nikhil, Artificial_Intelligence)
 $\forall x \text{ Student}(x) \rightarrow \text{UniversityStudent}(x)$
 $\forall x \text{ Student}(x) \rightarrow \text{HasID}(x)$
 $\forall x \text{ Studies}(x, \text{Artificial_Intelligence}) \rightarrow \text{AIStudent}(x)$